

PHẦN 1: KIẾN TRÚC HỆ THỐNG SERVER

1.1. Mô hình Asynchronous Event-Loop

Hệ thống Server được xây dựng theo mô hình **Asynchronous I/O (Bất đồng bộ)** sử dụng thư viện **Boost.Asio**, cho phép xử lý đồng thời nhiều kết nối mạng mà không bị tắc nghẽn (non-blocking). Điểm đặc biệt của Server này là khả năng **Hybrid Protocol Handling**: nó có thể xử lý cả giao thức **HTTP** (để tải file) và **WebSocket** (để điều khiển thời gian thực) trên cùng một cổng duy nhất (Port **9001**).

Server hoạt động đa luồng (Multi-threading) dựa trên số lõi CPU của phần cứng, đảm bảo hiệu năng cao khi chịu tải.

1.2. Phân tích cơ chế hoạt động

1.2.1. Quản lý vòng đời kết nối

Class **Listener** đóng vai trò là "người gác cổng". Quy trình tiếp nhận kết nối diễn ra như sau:

1. **Khởi tạo Socket**: Server bind vào địa chỉ **0.0.0.0** (lắng nghe mọi interface mạng) trên cổng **9001**.
2. **Thread Pool**: **io_context** được chạy trên số lượng luồng tương ứng với số nhân CPU (**std::thread::hardware_concurrency**). Điều này đảm bảo tận dụng tối đa phần cứng.
3. **Phát hiện giao thức**:
 - **Cơ chế Dual-Mode**: Một cổng duy nhất xử lý được cả hai việc: phục vụ file (**HTTP**) và nhận lệnh (**WebSocket**).
 - **Cơ chế**: Trong **Listener::on_accept**, khi có kết nối TCP, Server thực hiện "peek" (đọc thử) header của request.
 - **Xử lý**:
 - Nếu thấy **Upgrade: websocket**: Chuyển socket cho **WebSocketSession** (Kết nối điều khiển dài hạn).
 - Nếu là HTTP GET thường: Gọi **handle_http_file_request** để trả về file ảnh/video rồi đóng kết nối.

1.2.2. Phân Hệ WebSocket

Đây là "trái tim" của hệ thống, nơi duy trì kết nối thời gian thực.

- **Quản lý Session Độc quyền**:
 - Biến **active_ws_session** đảm bảo tại một thời điểm chỉ có duy nhất một Admin được quyền điều khiển.
 - Nếu Admin thứ hai cố kết nối, Server sẽ trả về HTTP 503 (Server Busy). Điều này tránh xung đột lệnh điều khiển.

- **Cơ chế Strand:**

- Sử dụng `net::strand` để bọc các handler. Điều này đảm bảo các callback (read/write) của cùng một socket không bao giờ chạy song song, loại bỏ hoàn toàn tình trạng Race Condition mà không cần dùng Mutex khóa toàn bộ socket.

- **Hàng đợi Gửi:**

- Do tính chất bất đồng bộ, ta không thể gọi lệnh `async_write` liên tiếp khi lệnh trước chưa xong.
- Class `WebsocketSession` cài đặt một `std::vector` làm hàng đợi (`queue_`). Nếu socket đang bận gửi, dữ liệu mới sẽ được xếp vào hàng chờ.

PHẦN 2: GIAO THỨC GIAO TIẾP (PROTOCOL)

Giao tiếp hoàn toàn dựa trên **JSON**.

Cấu trúc Request (Client -> Server):

```
{ "command": "webcam_record", "payload": { "duration": 5.0 } }
```

Cấu trúc Request (Server -> Client):

```
{ "command": "webcam_record", "payload": "/captures/video_recording.mp4" }
```

Các lệnh chính:

- `shutdown`, `restart`: Điều khiển nguồn.
- `keylogger_start`, `keylogger_stop`: Ghi phím.
- `webcam_capture`, `screen_capture`: Chụp ảnh.
- `list_processes`, `stop_process`: Quản lý tác vụ.

PHẦN 3: PHÂN TÍCH CHI TIẾT MODULES CHỨC NĂNG

3.1. Module điều khiển nguồn (`control.cpp`)

Chức năng: Dùng để **Shutdown**, hay **Restart** máy chủ nguồn (**Server**).

Kĩ thuật: Với kiến trúc máy **Server** giả định là hệ điều hành Windows.

- **Cơ chế:** Sử dụng `system()` để sử dụng các lệnh của hệ điều hành để điều khiển nguồn. Giữ delay 2s trước khi **Shutdown** hay **Restart** máy, để gửi phản hồi (response) cho **Client** trước khi bị ngắt kết nối.

Kết quả: Trả về `success` nếu thành công, hay `error` nếu thất bại và gửi `error_code` cho **Client**.

3.2. Module Keylogger (`keylogger.cpp`)

Chức năng: Dùng để ghi lại các phím bấm (key press), kể cả chuột, của máy phía **Server** cho đến khi nhận được yêu cầu dừng của bên phía **Client**.

Kĩ thuật: Đây là module phức tạp nhất về quản lý luồng (concurrency).

- **Vấn đề:** Việc quét phím yêu cầu vòng lặp vô hạn `while(true)`. Nếu chạy trên luồng mạng chính, **Server** sẽ bị treo.
- **Giải pháp:**
 - **Đa luồng:** Keylogger chạy trên một `std::thread` hoàn toàn riêng biệt (`keylogger_thread_`) để vòng lặp `while(true)` của nó không chặn luồng mạng chính. riêng biệt.
 - **Polling:** Sử dụng `GetAsyncKeyState` để bắt phím ở mức hệ thống (ngay cả khi ứng dụng không focus).
 - **Đồng bộ hóa:**
 - `std::atomic<bool> is_logging_`: Cờ hiệu để dừng vòng lặp an toàn.
 - `std::mutex`: Khóa bảo vệ dữ liệu log khi **Client** yêu cầu đọc, tránh xung đột dữ liệu (Race Condition).
- **Kết quả:** Trả về cho **Client** 1 chuỗi kí tự (string), biểu thị cho các phím bấm được ghi lại.

3.3. Module Webcam (`webcam.cpp`)

Chức năng: Sử dụng webcam để chụp hoặc quay trong 1 khoảng thời gian (duration) nhất định rồi trả về cho **Client** kết quả tương ứng.

Kĩ thuật: Sử dụng OpenCV để tương tác phần cứng.

- **Khởi tạo:** `VideoCapture(0, CAP_DSHOW)` giúp tăng tốc khởi động camera trên Windows.
- **Nén Video:** Sử dụng codec **H.264** (`VideoWriter::fourcc('H', '2', '6', '4')`). Việc nén này cực kỳ quan trọng để giảm dung lượng video truyền qua mạng (từ hàng GB xuống vài MB).
- **Timing:** Sử dụng `cv::getTickCount()` để đảm bảo thời gian quay video chính xác đến mili-giây.
- **Write:** Sử dụng `write()` (hay `imwrite()` cho `capture`) để ghi từng khung hình vào trong file.

Kết quả: Trả về cho **Client** đường dẫn tương đối với file `.exe` của **Server** (`/captures/anh_chup.jpg` cho ảnh, và `/captures/video_recording.mp4` cho video)